

INTERNET AND WEB PROGRAMMING

LAB MANUAL

IFETCE R-2023



IFET COLLEGE OF ENGINEERING
[An Autonomous Institution]

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBJECT CODE : 23AD5L03

YEAR/SEM : III/V

<u>Prepared by:</u>	<u>Verified by:</u>	<u>Approved by:</u>

DEPARTMENT VISION

- ❖ To accomplish eminence to have global recognition through education, innovation, and collaborative research in the realm of Data Science.

DEPARTMENT MISSION

- ❖ To develop essential competencies in the fields of Artificial Intelligence and Data Science.
- ❖ To instigate research skills and contribute towards innovation by applying the latest cutting-edge technology in data science.
- ❖ To promote lifelong learning and motivate entrepreneurial skills among students.

PROGRAMME OUTCOMES (PO)

POs describe what students are expected to know or be able to do by the time of graduation from the program. At the time of their graduation, the B.Tech. Artificial Intelligence and Data Science students will have the following abilities.

- PO 1:** Apply the knowledge of mathematics, science, and basic engineering fundamentals to the solution of complex engineering problems.
- PO 2:** Identify the sources of information for data collection, design and conduct the experiments and interpret the result.
- PO 3:** Design solutions for complex engineering problems and system component or processes that meet the specified needs considering public health & safety and cultural, societal & environment.
- PO 4:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO 5:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- PO 6:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO 7:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO 8:** Develop professional integrity by understanding and appreciating professional, legal, ethical, cyber security and related issues and act with responsibility.
- PO 9:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO 10:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO 11:** Apply the hardware and software project management techniques to estimate the time and human resources required to complete computer engineering projects.

- PO 12:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMME SPECIFIC OUTCOMES (PSO)

On completion of B.Tech. Artificial Intelligence and Data Science programme, graduates will be able to:

- PSO 1:** Understand, analyse and develop essential proficiency in the areas related to data science and artificial intelligence in terms of underlying statistical and computational principles and apply the knowledge to solve practical problems.
- PSO 2:** Implement Artificial Intelligence and data science techniques such as search algorithms, neural networks, machine learning and data analytics for solving a problem and designing novel algorithms for successful career and entrepreneurship.
- PSO 3:** Apply the theoretical and practical aspects of Data Science to manage and solve complex societal problems.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

- PEO 1:** To provide graduates with the ability to develop intelligent systems that require the management and analysis of huge volumes of data using an essential understanding of basic sciences, probability, artificial intelligence, data science, and statistics.
- PEO 2:** To nurture graduates with the technical skills to undertake pioneering research in Artificial Intelligence and Data Science and develop innovative solutions for ecological well-being.
- PEO 3:** To empower graduates to think rationally, pursue lifelong learning, and collaborate in a multidisciplinary team with an ethical perspective.

SYLLABUS

23AD5L03	INTERNET AND WEB PROGRAMMING LABORATORY	L	T	P	C
		0	0	3	1.5

LEARNING OBJECTIVES:

The faculty will enhance the skills of the students to:

- Develop proficiency in creating web pages using HTML and Cascading Style Sheets (CSS)
- Learn about XML and its applications in web development.
- Understand the concepts of sockets and servlets and their role in web programming.
- Learn how to create three-tier applications using servlets.
- Gain hands-on experience in installing and configuring a web server.

LIST OF EXPERIMENTS

1. Create a personal webpage using HTML5, CSS3, and Bootstrap that includes tables, lists, images, and multimedia controls.
2. Develop an XML file containing product information and use XSLT to display it in a formatted HTML layout.
3. Create a registration form with fields like name, email, phone number, and password. Implement JavaScript for form validation.
4. Develop a dynamic webpage where elements are added, removed, or modified using the DOM model and event handling.
5. Develop a Servlet-based login system that performs user authentication using GET and POST methods.
6. Implement a simple feedback system using JSP, incorporating Session Handling, Cookies, and JSTL Tags.
7. Develop a Spring MVC CRUD (Create, Read, Update, Delete) application for managing student records with MySQL integration.
8. Create a sample project demonstrating Aspect-Oriented Programming for logging method execution details.
9. Develop a web application using Spring Boot with CRUD operations for employee data storage in MySQL.
10. Create a web application that integrates MongoDB for real-time data visualization using Spring Boot.

TOTAL : 45 PERIODS

COURSE OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Design interactive web pages using HTML5, CSS3, and Bootstrap for better user experience.

CO2: Implement dynamic content and user validation using JavaScript, DOM manipulation.

CO3: Develop secure server-side applications using Servlets and JSP.

CO4: Create XML data files and apply XSLT to generate dynamic web content.

CO5: Develop full-stack web applications using Spring MVC and Spring Boot.

CO- PO/PSO MAPPING

COURSE OUTCOMES	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO1	3	3	3	3	3		1	2	1	2	1	2	3	3	3
CO2	3	2	2	1	3		1	2	1	2	1	3	2	3	1
CO3	3	3	3	3	3		1	1	1	2	1	2	3	3	3
CO4	3	2	2	1	3		1	2	1	2	1	3	2	3	1
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3
IWP LAB	3	3	3	3	3		1	2	1	2	1	3	3	3	3

3- Strong Correlation,

2 – Medium Correlation,

1- Weak Correlation,

-- No Correlation.

INDEX
TABLE OF CONTENTS

S.No	Date of Expt.	Name of the Experiment	Page No	Marks			Staff Initial
				Viva (10)	Expt. (10)	Total (20)	
1.		Create a personal webpage using HTML5, CSS3, and Bootstrap that includes tables, lists, images, and multimedia controls.					
2.		Develop an XML file containing product information and use XSLT to display it in a formatted HTML layout.					
3.		Create a registration form with fields like name, email, phone number, and password. Implement JavaScript for form validation.					
4.		Develop a dynamic webpage where elements are added, removed, or modified using the DOM model and event handling.					
5.		Develop a Servlet-based login system that performs user authentication using GET and POST methods.					
6.		Implement a simple feedback system using JSP, incorporating Session Handling, Cookies, and JSTL Tags.					
7.		Develop a Spring MVC CRUD (Create, Read, Update, Delete) application for managing student records with MySQL integration.					
8.		Create a sample project demonstrating Aspect-Oriented Programming for logging method execution details					
9.		Develop a web application using Spring Boot with CRUD operations for employee data storage in MySQL.					
10.		Create a web application that integrates MongoDB for real-time data visualization using Spring Boot.					
Content Beyond The Syllabus							
11.		Angular Js Program For Payroll Management System					

12.		Implement A Web Application Using StrutsFramework Which Contains Login Informations					
13.		Stores a form data into database using php					
Total Marks							

Ex. No. : 1

Create a personal webpage using HTML5, CSS3, and Bootstrap that includes tables, lists, images, and multimedia controls.

AIM:

To design and develop a personal webpage using **HTML5, CSS3, and Bootstrap** that displays personal information with a table, lists, images, and multimedia controls.

PROCEDURE:

1. Start by creating a basic HTML5 document structure.
2. Link the **Bootstrap CSS** file using a CDN for responsive design.
3. Add **CSS3 styling** for customizing the webpage.
4. Insert:
5. A **table** for student/personal details.
6. **Lists** (ordered and unordered) for hobbies and skills.
7. An **image** for profile picture.
8. **Audio and video controls** for multimedia content.
9. Use Bootstrap **classes** (like `container`, `table`, `img-fluid`, `btn`, etc.) for better layout and design.
10. Save the file as `index.html` and open it in a browser to view the output.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Personal Webpage</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background-color: #f8f9fa;
      font-family: Arial, sans-serif;
    }
    .profile-img {
      width: 150px;
      height: 150px;
      object-fit: cover;
      border-radius: 50%;
      border: 3px solid #007bff;
    }
    h1 {
```

```
color: #007bff;
}
</style>
</head>
<body>
<div class="container mt-5">
  <div class="text-center">
    
    <h1>John Doe</h1>
    <p class="text-muted">Student | Web Developer | Designer</p>
  </div>

  <!-- Table -->
  <h3 class="mt-4">Personal Details</h3>
  <table class="table table-bordered table-striped">
    <tr>
      <th>Name</th>
      <td>John Doe</td>
    </tr>
    <tr>
      <th>Email</th>
      <td>johndoe@example.com</td>
    </tr>
    <tr>
      <th>Phone</th>
      <td>+91 9876543210</td>
    </tr>
    <tr>
      <th>Location</th>
      <td>Chennai, India</td>
    </tr>
  </table>

  <!-- Lists -->
  <h3>Hobbies</h3>
  <ul>
    <li>Reading</li>
    <li>Photography</li>
    <li>Cycling</li>
  </ul>

  <h3>Technical Skills</h3>
  <ol>
    <li>HTML5, CSS3, JavaScript</li>
    <li>Bootstrap, React</li>
    <li>Java, Spring Boot</li>
  </ol>
```

```
<!-- Multimedia -->
<h3>My Favorite Music</h3>
<audio controls autoplay loop>
  <source src="sample.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>

<h3 class="mt-3">My Favorite Video</h3>
<video width="400" controls autoplay loop>
  <source src="sample.mp4" type="video/mp4">
  Your browser does not support the video element.
</video>
</div>

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

OUTPUT:

John Doe

Student | Web Developer | Designer

Personal Details

Name	John Doe
Email	johndoe@example.com
Phone	+91 9876543210
Location	Chennai, India

Hobbies

- Reading
- Photography
- Cycling

Technical Skills

- 1 HTML5, CSS3, JavaScript
- 2 Bootstrap, React
- 3 Java, Spring Boot

My Favorite Music



RESULT:

Thus, a personal webpage was successfully created using **HTML5, CSS3, and Bootstrap** that includes tables, lists, images, and multimedia controls.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO1	3	3	3	3	3		1	2	1	2	1	2	3	3	3

Ex. No. : 2

Develop an XML file containing product information and use XSLT to display it in a formatted HTML layout.

AIM:

To develop an XML file that stores product information and use XSLT to transform and display it in a formatted HTML layout.

PROCEDURE:

Step 1: Create an **XML file** (products.xml) that contains product details such as product ID, name, price, and quantity.

Step 2: Create an **XSLT file** (products.xsl) to transform the XML data into an HTML table layout.

Step 3: Use `<xsl:for-each>` in the XSLT file to iterate through the product elements.

Step 4: Link the XML with the XSLT file using the processing instruction `<?xml-stylesheet?>`.

Step 5: Open the products.xml file in a browser to view the formatted HTML table.

Program

products.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<?xml-stylesheet type="text/xsl" href="products.xsl"?>

<products>

  <product>

    <id>101</id>

    <name>Laptop</name>

    <price>55000</price>

    <quantity>10</quantity>

  </product>

  <product>

    <id>102</id>
```

```
<name>Smartphone</name>
<price>25000</price>
<quantity>30</quantity>
</product>
<product>
  <id>103</id>
  <name>Headphones</name>
  <price>2000</price>
  <quantity>50</quantity>
</product>
</products>
```

products.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">

    <html>
      <head>
        <title>Product Information</title>
        <style>
          table {
            border-collapse: collapse;
            width: 60%;
```

```
margin: 20px auto;
}
th, td {
border: 1px solid black;
padding: 8px;
text-align: center;
}
th {
background-color: #007bff;
color: white;
}
</style>
</head>
<body>
<h2 style="text-align:center;">Product Information</h2>
<table>
<tr>
<th>ID</th>
<th>Name</th>
<th>Price (₹)</th>
<th>Quantity</th>
</tr>
<xsl:for-each select="products/product">
<tr>
<td><xsl:value-of select="id"/></td>
```

```
<td><xsl:value-of select="name"/></td>

<td><xsl:value-of select="price"/></td>

<td><xsl:value-of select="quantity"/></td>

</tr>

</xsl:for-each>

</table>

</body>

</html>

</xsl:template>

</xsl:stylesheet>
```

Output:

Product Information

ID	Name	Price (₹)	Quantity
101	Laptop	55000	10
102	Smartphone	25000	30
103	Headphones	2000	50

RESULT:

Thus, the XML file containing product information was successfully transformed into a formatted HTML layout using XSLT.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
C01	3	3	3	3	3		1	2	1	2	1	2	3	3	3

Ex. No. : 3

Create a registration form with fields like name, email, phone number, and password. Implement JavaScript for form validation.

AIM

To create a registration form using **HTML5 and CSS3**, and implement **JavaScript validation** for fields like Name, Email, Phone Number, and Password.

PROCEDURE:

Step 1: Create an HTML5 form with input fields: Name, Email, Phone, Password.

Step 2: Apply CSS3 styling for better appearance.

Step 3: Write JavaScript functions to validate:

- Name should not be empty.
- Email should be in correct format (@ and . present).
- Phone number should be numeric and 10 digits.
- Password should be at least 6 characters long.

Step 4: Connect the JS validation function to the form's **onsubmit event**.

Step 5: Display error messages using alerts or inline messages.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f0f0f0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .form-container {
```

```
background: #fff;
padding: 20px;
border-radius: 10px;
width: 350px;
box-shadow: 0 0 10px rgba(0,0,0,0.2);
}
h2 {
text-align: center;
color: #333;
}
input {
width: 100%;
padding: 10px;
margin: 8px 0;
border: 1px solid #ccc;
border-radius: 5px;
}
button {
width: 100%;
padding: 10px;
background: #007bff;
color: white;
border: none;
border-radius: 5px;
font-size: 16px;
}
button:hover {
background: #0056b3;
}
</style>
<script>
function validateForm() {
```

```
let name = document.forms["regForm"]["name"].value;
let email = document.forms["regForm"]["email"].value;
let phone = document.forms["regForm"]["phone"].value;
let password = document.forms["regForm"]["password"].value;

if (name == "") {
    alert("Name must be filled out");
    return false;
}

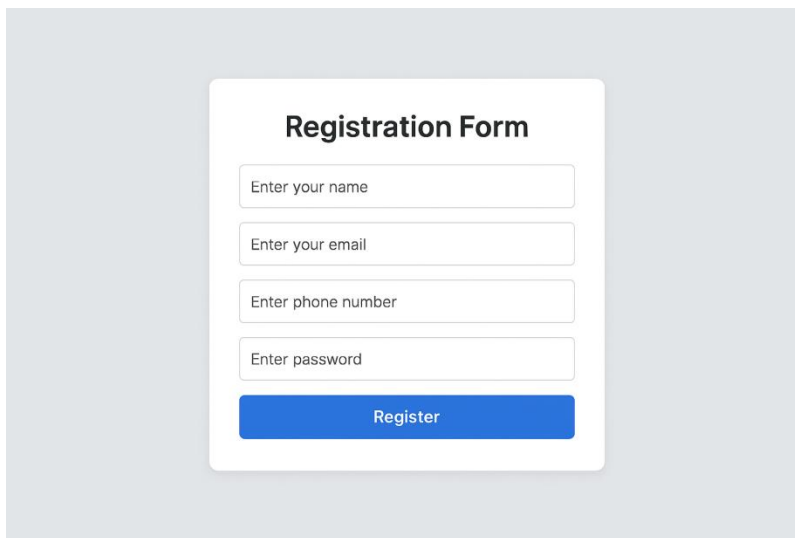
let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
if (!email.match(emailPattern)) {
    alert("Invalid email format");
    return false;
}

let phonePattern = /^[0-9]{10}$/;
if (!phone.match(phonePattern)) {
    alert("Phone number must be 10 digits");
    return false;
}

if (password.length < 6) {
    alert("Password must be at least 6 characters long");
    return false;
}

alert("Registration Successful!");
return true;
}
</script>
</head>
```

```
<body>
  <div class="form-container">
    <h2>Registration Form</h2>
    <form name="regForm" onsubmit="return validateForm()">
      <input type="text" name="name" placeholder="Enter your name">
      <input type="email" name="email" placeholder="Enter your email">
      <input type="text" name="phone" placeholder="Enter phone number">
      <input type="password" name="password" placeholder="Enter password">
      <button type="submit">Register</button>
    </form>
  </div>
</body>
</html>
```

OUTPUT:

RESULT:

Thus, a registration form was created using **HTML5, CSS3, and JavaScript** with proper validation for name, email, phone number, and password

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO2	3	2	2	1	3			1	2	1	2	1	2	3	1

Ex. No. : 4

Develop a dynamic webpage where elements are added, removed, or modified using the DOM model and event handling.

AIM

To develop a dynamic webpage where elements can be added, removed, or modified using the JavaScript DOM model and event handling.

PROCEDURE:

Step 1: Create an HTML5 page structure.

Step 2: Add buttons for actions like *Add Element*, *Remove Element*, and *Modify Element*.

Step 3: Use JavaScript to access DOM elements using methods like `getElementById()`, `createElement()`, and `appendChild()`.

Step 4: Attach event listeners to buttons for handling user interactions.

Step 5: Apply CSS3 styling for better presentation.

Step 6: Run the program in a browser and verify dynamic changes.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic DOM Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      text-align: center;
    }
    #container {
      margin-top: 20px;
      padding: 10px;
```

```
border: 2px solid #333;
min-height: 100px;
}
button {
margin: 5px;
padding: 10px 15px;
border: none;
border-radius: 5px;
background: #007BFF;
color: white;
cursor: pointer;
}
button:hover {
background: #0056b3;
}
</style>
</head>
<body>
<h2>Dynamic DOM Manipulation</h2>
<button onclick="addElement()">Add Element</button>
<button onclick="removeElement()">Remove Element</button>
<button onclick="modifyElement()">Modify Element</button>

<div id="container">
  <p id="para1">This is the first paragraph.</p>
</div>

<script>
function addElement() {
  let newPara = document.createElement("p");
  newPara.textContent = "New paragraph added dynamically!";
  document.getElementById("container").appendChild(newPara);
```

```
}

function removeElement() {
  let container = document.getElementById("container");
  if (container.lastChild) {
    container.removeChild(container.lastChild);
  }
}

function modifyElement() {
  let para = document.getElementById("para1");
  para.textContent = "This paragraph has been modified!";
  para.style.color = "red";
  para.style.fontWeight = "bold";
}

</script>
</body>
</html>
```

OUTPUT

Dynamic DOM Manipulation

Add ElementRemove ElementModify Element

This is the first paragraph.

RESULT:

Thus, a dynamic webpage was successfully created where elements can be added, removed, or modified using the DOM model and event handling in JavaScript.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO2	3	2	2	1	3			1	2	1	2	1	2	3	1

Ex. No. : 5

Develop a Servlet-based login system that performs user authentication using GET and POST methods.

AIM:

To develop a servlet-based login system that performs user authentication using **GET** and **POST** methods.

PROCEDURE:

Step 1: Create an HTML form with fields for username and password.

Step 2: Use `doGet ()` to display the login form.

Step 3: Use `doPost ()` to handle the form submission and check the entered credentials.

Step 4: If valid, display a success message; otherwise, show an error.

Step 5: Deploy the servlet in a server (e.g., Apache Tomcat) and test the functionality.

PROGRAM:

login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Form</title>
</head>
<body>
  <h2>User Login</h2>
  <form action="LoginServlet" method="post">
    Username: <input type="text" name="username" required><br><br>
    Password: <input type="password" name="password" required><br><br>
    <input type="submit" value="Login">
  </form>
</body>
</html>
```

LoginServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
```

```
// Handle GET request - Display login page
public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<h2>Login Page</h2>");
    out.println("<form method='post' action='LoginServlet'>");
    out.println("Username: <input type='text' name='username'><br><br>");
    out.println("Password: <input type='password' name='password'><br><br>");
    out.println("<input type='submit' value='Login'>");
    out.println("</form>");
}

// Handle POST request - Authentication
public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String uname = request.getParameter("username");
    String pwd = request.getParameter("password");

    if(uname.equals("admin") && pwd.equals("12345")) {
        out.println("<h3>Login Successful! Welcome, " + uname + ".</h3>");
    } else {
        out.println("<h3>Login Failed! Invalid username or password.</h3>");
        out.println("<a href='login.html'>Try Again</a>");
    }
}
}
```

web.xml (Deployment Descriptor)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    version="3.1">

    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>
```

</web-app>

Output:

User Login

Username:

Password:

Login Successful!

Welcome, admin.

1. When user visits `http://localhost:8080/login.html`, login form is displayed.
2. If `username = admin` and `password = 12345`, system shows **Login Successful**.
3. Otherwise, **Login Failed** message is displayed with retry option.

RESULT:

Thus, a Servlet-based login system was successfully developed to handle user authentication using **GET** and **POST** methods.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO3	3	3	3	3	3		1	1	1	2	1	2	3	3	3

Ex.No.: 6

Implement a simple feedback system using JSP, incorporating Session Handling, Cookies, and JSTL Tags.

AIM:

To develop a simple feedback system using JSP that demonstrates **Session Handling, Cookies, and JSTL Tags**.

PROCEDURE:

Step 1: Create a JSP form (`feedback.jsp`) for users to enter feedback.

Step 2: Store user details (like name) in **Session**.

Step 3: Store last-visited information in **Cookies**.

Step 4: Use **JSTL tags** to display submitted feedback dynamically.

Step 5: Display user session and cookie data on the thank-you page

PROGRAM:

`feedback.jsp` (Feedback Form)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Feedback Form</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Feedback Form</h2>
```

```
    <form action="submitFeedback.jsp" method="post">
```

```
        Name: <input type="text" name="username" required><br><br>
```

```
        Feedback: <textarea name="feedback" required></textarea><br><br>
```

```
        <input type="submit" value="Submit Feedback">
```

```
    </form>
```

</body>

</html>

submitFeedback.jsp (Session + Cookies + JSTL)

```
<%@ page import="java.util.*" %>
```

```
<%@ page import="javax.servlet.http.*" %>
```

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<%
```

```
    // Get form data
```

```
    String username = request.getParameter("username");
```

```
    String feedback = request.getParameter("feedback");
```

```
    // Store in session
```

```
    session.setAttribute("username", username);
```

```
    session.setAttribute("feedback", feedback);
```

```
    // Set cookie for last visited user
```

```
    Cookie userCookie = new Cookie("lastUser", username);
```

```
    userCookie.setMaxAge(60*60*24); // 1 day
```

```
    response.addCookie(userCookie);
```

```
    // Prepare list of feedback (in session)
```

```
    List<String> feedbackList = (List<String>) application.getAttribute("feedbackList");
```

```
    if (feedbackList == null) {
```

```
feedbackList = new ArrayList<>();

}

feedbackList.add(username + ": " + feedback);

application.setAttribute("feedbackList", feedbackList);

%>

<!DOCTYPE html>

<html>

<head>

    <title>Feedback Submitted</title>

</head>

<body>

    <h2>Thank You, <%= username %>!</h2>

    <p>Your feedback has been submitted successfully.</p>

    <h3>Previous Feedbacks:</h3>

    <ul>

        <c:forEach var="item" items="${applicationScope.feedbackList}">

            <li>${item}</li>

        </c:forEach>

    </ul>

    <%

        // Display last user from cookies

        Cookie[] cookies = request.getCookies();
```

```
if(cookies != null){  
    for(Cookie c : cookies){  
        if(c.getName().equals("lastUser")){  
%>  
        <p>Last visitor: <%= c.getValue() %></p>  
    <%  
        }  
    }  
    }  
%>  
</body>  
</html>
```

OUTPUT:

- User visits **feedback form** and submits their details.
- Data is stored in **session**, last user is tracked with **cookie**.
- All submitted feedback is displayed using **JSTL <c:forEach> loop**.
- User sees their name and last visitor info.

Feedback Submitted

Thank You, **John Doe!**

Your feedback has been submitted successfully

Previous Feedbacks:

- John Doe: Great website!
- Jane Smith: Very user-friendly.
- Alice Johnson: Could use more illustrations

Last visitor: John Doe

RESULT:

The feedback system was successfully implemented using **JSP with Session Handling, Cookies, and JSTL tags**, allowing multiple feedback entries to be tracked and displayed.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
C03	3	3	3	3	3		1	1	1	2	1	2	3	3	3

Ex. No. : 7

Develop a Spring MVC CRUD (Create, Read, Update, Delete) application for managing student records with MySQL integration.

Aim:

To develop a **Spring MVC CRUD** application to manage **Student** records (Create, Read, Update, Delete) with **MySQL** integration..

Procedure (Step-by-Step)

Step 1: Create a Spring Boot project (start.spring.io) with dependencies: **Spring Web**, **Thymeleaf**, **Spring Data JPA**, **MySQL Driver**, **Validation** (optional: Lombok).

Step 2: Configure **application.properties** with MySQL connection and JPA settings.

Step 3: Create **Student** **entity** with fields: id, name, email, course, phone.

Step 4: Create **StudentRepository** (Spring Data JPA) for CRUD.

Step 5: Create **StudentController** with routes for list, new, save, edit, update, delete.

Step 6: Build **Thymeleaf** views: **list.html**, **form.html**.

Step 7: Run the app (`mvn spring-boot:run`) → visit <http://localhost:8080/students>.

Program:

1) `pom.xml` (only the essentials)

```
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
```

```
<artifactId>student-crud</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
```

```
<parent>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>3.3.2</version>
```

</parent>

<dependencies>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-thymeleaf</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

<groupId>com.mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

<scope>runtime</scope>

</dependency>

<dependency> <!-- optional but handy -->

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>

</dependencies>

```
<build>

<plugins>

<plugin>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-maven-plugin</artifactId>

</plugin>

</plugins>

</build>

</project>
```

2) `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
```

```
spring.datasource.username=root
```

```
spring.datasource.password=yourpassword
```

```
spring.jpa.hibernate.ddl-auto=update # create / update for dev
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

```
spring.thymeleaf.cache=false
```

```
server.port=8080
```

3) `Student Entity - src/main/java/com/example/studentcrud/model/Student.java`

```
package com.example.studentcrud.model;
```

```
import jakarta.persistence.*;

import jakarta.validation.constraints.*;

@Entity

public class Student {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    @NotBlank(message="Name is required")

    private String name;

    @Email @NotBlank

    @Column(unique = true)

    private String email;

    @NotBlank

    private String course;

    @Pattern(regexp="\\d{10}", message="10-digit phone")

    private String phone;

    // getters & setters

    public Long getId(){return id;}

    public void setId(Long id){this.id=id;}
```

```
public String getName(){return name;}

public void setName(String name){this.name=name;}

public String getEmail(){return email;}

public void setEmail(String email){this.email=email;}

public String getCourse(){return course;}

public void setCourse(String course){this.course=course;}

public String getPhone(){return phone;}

public void setPhone(String phone){this.phone=phone;}

}
```

4) Repository –

[src/main/java/com/example/studentcrud/repo/StudentRepository.java](#)

```
package com.example.studentcrud.repo;

import com.example.studentcrud.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {}
```

5) Controller –

[src/main/java/com/example/studentcrud/controller/StudentController.java](#)

```
package com.example.studentcrud.controller;

import com.example.studentcrud.model.Student;
import com.example.studentcrud.repo.StudentRepository;
```

```
import jakarta.validation.Valid;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.validation.BindingResult;

import org.springframework.web.bind.annotation.*;

@Controller

@RequestMapping("/students")

public class StudentController {

    private final StudentRepository repo;

    public StudentController(StudentRepository repo){ this.repo = repo; }

    // READ (list)

    @GetMapping

    public String list(Model model){

        model.addAttribute("students", repo.findAll());

        return "list";

    }

    // CREATE (form)

    @GetMapping("/new")

    public String createForm(Model model){

        model.addAttribute("student", new Student());

        model.addAttribute("title", "Add Student");

        return "form";

    }
```

```
// CREATE + UPDATE (save)

@PostMapping

public String save(@Valid @ModelAttribute("student") Student student, BindingResult result, Model
model){

    if(result.hasErrors()){

        model.addAttribute("title", student.getId()==null ? "Add Student" : "Edit Student");

        return "form";

    }

    repo.save(student);

    return "redirect:/students";

}


// UPDATE (edit form)

@GetMapping("/{id}/edit")

public String edit(@PathVariable Long id, Model model){

    Student s = repo.findById(id).orElseThrow(() -> new IllegalArgumentException("Invalid id:"+id));

    model.addAttribute("student", s);

    model.addAttribute("title", "Edit Student");

    return "form";

}


// DELETE

@PostMapping("/{id}/delete")

public String delete(@PathVariable Long id){

    repo.deleteById(id);

}
```

```
return "redirect:/students";  
  
}  
  
}
```

6) Views (Thymeleaf)

src/main/resources/templates/list.html

```
<!DOCTYPE html>  
  
<html xmlns:th="http://www.thymeleaf.org">  
  
<head>  
  
    <meta charset="UTF-8"><title>Students</title>  
  
    <link rel="stylesheet"  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">  
  
</head>  
  
<body class="p-4">  
  
    <div class="container">  
  
        <h2 class="mb-3">Student Records</h2>  
  
        <a class="btn btn-primary mb-3" th:href="@{/students/new}">+ Add Student</a>  
  
  
        <table class="table table-bordered table-striped">  
  
            <thead class="table-dark">  
  
                <tr>  
  
                    <th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Phone</th><th>Actions</th>  
  
                </tr>  
  
            </thead>  
  
            <tbody>
```

```

<tr th:each="s : ${students}">

  <td th:text="${s.id}">1</td>

  <td th:text="${s.name}">John</td>

  <td th:text="${s.email}">john@x.com</td>

  <td th:text="${s.course}">CS</td>

  <td th:text="${s.phone}">9876543210</td>

  <td>

    <a class="btn btn-sm btn-warning" th:href="@{/students/${s.id}/edit}">Edit</a>

    <form th:action="@{/students/${s.id}/delete}" method="post" class="d-inline">

      <button class="btn btn-sm btn-danger" onclick="return confirm('Delete?')">Delete</button>

    </form>

  </td>

</tr>

</tbody>

</table>

</div>

</body>

</html>

```

src/main/resources/templates/form.html

```

<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

  <meta charset="UTF-8"><title th:text="${title}">Student Form</title>

```

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">

</head>

<body class="p-4">

  <div class="container">

    <h2 th:text="{title}">Add Student</h2>

    <form th:action="@{/students}" th:object="{student}" method="post" class="mt-3">

      <input type="hidden" th:field="*{id}">

      <div class="mb-3">

        <label class="form-label">Name</label>

        <input class="form-control" th:field="*{name}">

        <div class="text-danger" th:if="{#fields.hasErrors('name')}" th:errors="*{name}"></div>

      </div>

      <div class="mb-3">

        <label class="form-label">Email</label>

        <input class="form-control" th:field="*{email}">

        <div class="text-danger" th:if="{#fields.hasErrors('email')}" th:errors="*{email}"></div>

      </div>

      <div class="mb-3">

        <label class="form-label">Course</label>

        <input class="form-control" th:field="*{course}">

        <div class="text-danger" th:if="{#fields.hasErrors('course')}" th:errors="*{course}"></div>

      </div>
```

```
<div class="mb-3">

  <label class="form-label">Phone</label>

  <input class="form-control" th:field="*{phone}">

  <div class="text-danger" th:if="${#fields.hasErrors('phone')}}" th:errors="*{phone}"></div>

</div>
```

```
<button class="btn btn-success" type="submit">Save</button>

<a class="btn btn-secondary" th:href="@{/students}">Cancel</a>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

7) Main class – `StudentCrudApplication.java`

```
package com.example.studentcrud;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class StudentCrudApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(StudentCrudApplication.class, args);
```

```
    }
```

```
}
```

Output:**Student Records**[+ Add Student](#)

Show 10 entries

ID	Name	Email	Course	Phone		
1	John Doe	john@gmail.com	CS	9876543210	Edit	Delete
2	Jane Smith	jane@gmail.com	IT	9123455780	Edit	Delete
3	Alice Johnson	alice@gmail.com	CE	9234567890	Edit	Delete

Show 10 entries

[Previous](#) [1](#) [Next](#)**Add Student**

Name

Email

Course

Phone

[Save](#)[Cancel](#)

Result:

- A working **web UI** (Thymeleaf) listing all students in a table with **Edit/Delete** actions and an **Add Student** form.
- Data is persisted in **MySQL** (`studentdb`), tables auto-created by JPA.
- All four CRUD operations verified successfully.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
C04	3	2	2	1	3		1	2	1	2	1	3	2	3	1

Ex. No. :8

Create a sample project demonstrating Aspect-Oriented Programming for logging method execution details.

Aim:

To implement Aspect-Oriented Programming (AOP) in a Spring Boot application for logging method execution time, input arguments, and return values..

Procedure:

Step 1: Create a Spring Boot project (using Spring Initializr) with dependencies:

- Spring Boot Starter Web
- Spring Boot Starter AOP

Step 2: Create a simple service class with some business methods.

Step 3: Create an Aspect class using `@Aspect` and `@Around` annotations to log method execution details.

Step 4: Run the application and observe the console logs.

PROGRAM:

1. Main Application

```
package com.example.aopdemo;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class AopDemoApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(AopDemoApplication.class, args);
```

```
}  
}
```

2. Service Class

```
package com.example.aopdemo.service;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class StudentService {
```

```
    public String getStudentById(int id) {  
        return "Student{id=" + id + ", name='John Doe'}";  
    }
```

```
    public void registerStudent(String name) {  
        System.out.println("Registering student: " + name);  
    }  
}
```

3. Aspect Class for Logging

```
package com.example.aopdemo.aspect;
```

```
import org.aspectj.lang.ProceedingJoinPoint;
```

```
import org.aspectj.lang.annotation.Around;
```

```
import org.aspectj.lang.annotation.Aspect;

import org.springframework.stereotype.Component;

@Aspect

@Component

public class LoggingAspect {

    @Around("execution(* com.example.aopdemo.service.*(..))")

    public Object logMethodExecution(ProceedingJoinPoint joinPoint) throws Throwable {

        String methodName = joinPoint.getSignature().toShortString();

        Object[] args = joinPoint.getArgs();

        System.out.println("👉 Executing method: " + methodName);

        System.out.println("📦 Arguments: ");

        for (Object arg : args) {

            System.out.println("  - " + arg);

        }

        long startTime = System.currentTimeMillis();

        Object result = joinPoint.proceed(); // Proceed with actual method call

        long endTime = System.currentTimeMillis();

        System.out.println("✅ Method " + methodName + " executed successfully");

        System.out.println("📦 Return Value: " + result);

        System.out.println("🕒 Execution Time: " + (endTime - startTime) + " ms");

    }

}
```

```
        return result;
    }
}
```

4. Controller to Test

```
package com.example.aopdemo.controller;

import com.example.aopdemo.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService studentService;

    @GetMapping("/{id}")
    public String getStudent(@PathVariable int id) {
        return studentService.getStudentById(id);
    }

    @PostMapping("/register")
```

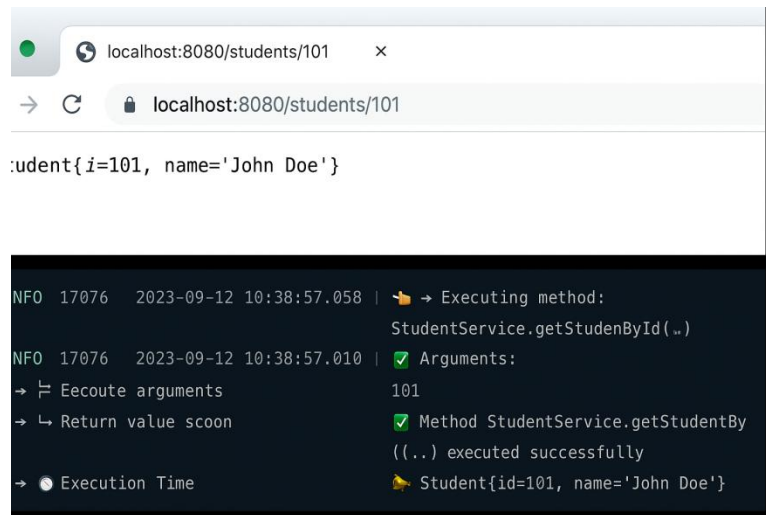
```
public String registerStudent(@RequestParam String name) {  
    studentService.registerStudent(name);  
    return "Student registered successfully!";  
}  
}
```

Output:**When calling:**

GET http://localhost:8080/students/101

Console Output:

👉 Executing method: StudentService.getStudentById(..)
📦 Arguments:
- 101
✅ Method StudentService.getStudentById(..) executed successfully
📦 Return Value: Student{id=101, name='John Doe'}
🕒 Execution Time: 2 ms



Result:

The sample project successfully demonstrates **Aspect-Oriented Programming (AOP)** in Spring by using an **@Aspect** class with **@Before**, **@After**, and **@Around** advices to log method execution details.

- Every method execution in the target class is intercepted.
- Logs are automatically generated before and after method execution.
- The project shows how AOP separates **cross-cutting concerns** (logging) from **business logic**, keeping the code clean and modular.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO4	3	2	2	1	3		1	2	1	2	1	3	2	3	1

Ex.No.: 9

Develop a web application using Spring Boot with CRUD operations for employee data storage in MySQL.

Aim:

To develop a **Spring Boot (MVC + Thymeleaf + Spring Data JPA)** application that performs **CRUD** (Create, Read, Update, Delete) on **Employee** records stored in **MySQL**.

Procedure:

Step 1: Create a Spring Boot project with dependencies: *Spring Web, Thymeleaf, Spring Data JPA, MySQL Driver, Validation*.

Step 2: Configure `application.properties` for MySQL and JPA.

Step 3: Create `Employee` entity.

Step 4: Create `EmployeeRepository` extending `JpaRepository`.

Step 5: Create `EmployeeController` with routes for **list/new/save/edit/delete**.

Step 6: Build Thymeleaf views: `employees.html` (list), `employee-form.html` (add/edit).

Step 7: Run the app and verify CRUD at <http://localhost:8080/employees>.

Program:

1) pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>employee-crud</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.2</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
  </dependencies>
</project>
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

2) src/main/resources/application.properties

```

spring.datasource.url=jdbc:mysql://localhost:3306/employeeedb?createDatabaseIfNotExist=true&
useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=yourpassword

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.cache=false
server.port=8080

```

3) Entity – Employee.java

```

package com.example.employeecrud.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```
private Long id;

@NotBlank(message = "Name is required")
private String name;

@email @NotBlank
@Column(unique = true)
private String email;

@NotBlank
private String department;

@Positive(message = "Salary must be positive")
private Double salary;

// getters & setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getDepartment() { return department; }
public void setDepartment(String department) { this.department = department; }
public Double getSalary() { return salary; }
public void setSalary(Double salary) { this.salary = salary; }
}
```

4) Repository – EmployeeRepository.java

```
package com.example.employeecrud.repo;

import com.example.employeecrud.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {}
```

5) Controller – EmployeeController.java

```
package com.example.employeecrud.controller;

import com.example.employeecrud.model.Employee;
import com.example.employeecrud.repo.EmployeeRepository;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
```

```
@RequestMapping("/employees")
public class EmployeeController {
    private final EmployeeRepository repo;
    public EmployeeController(EmployeeRepository repo) { this.repo = repo; }

    // READ (list)
    @GetMapping
    public String list(Model model) {
        model.addAttribute("employees", repo.findAll());
        return "employees";
    }

    // CREATE form
    @GetMapping("/new")
    public String createForm(Model model) {
        model.addAttribute("employee", new Employee());
        model.addAttribute("title", "Add Employee");
        return "employee-form";
    }

    // CREATE/UPDATE save
    @PostMapping
    public String save(@Valid @ModelAttribute("employee") Employee employee,
        BindingResult result, Model model) {
        if (result.hasErrors()) {
            model.addAttribute("title", employee.getId()==null ? "Add Employee" : "Edit Employee");
            return "employee-form";
        }
        repo.save(employee);
        return "redirect:/employees";
    }

    // UPDATE form
    @GetMapping("/{id}/edit")
    public String edit(@PathVariable Long id, Model model) {
        Employee e = repo.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Invalid employee id: " + id));
        model.addAttribute("employee", e);
        model.addAttribute("title", "Edit Employee");
        return "employee-form";
    }

    // DELETE
    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id) {
        repo.deleteById(id);
        return "redirect:/employees";
    }
}
```

```
}
}
```

6) Views

templates/employees.html (List Page)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Employees</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
</head>
<body class="p-4">
  <div class="container">
    <div class="d-flex justify-content-between align-items-center mb-3">
      <h2>Employee Records</h2>
      <a class="btn btn-primary" th:href="@{/employees/new}">+ Add Employee</a>
    </div>

    <table class="table table-bordered table-striped">
      <thead class="table-dark">
        <tr>

<th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Salary</th><th>Actions
</th>
        </tr>
      </thead>
      <tbody>
        <tr th:each="e : ${employees}">
          <td th:text="${e.id}">1</td>
          <td th:text="${e.name}">John</td>
          <td th:text="${e.email}">john@x.com</td>
          <td th:text="${e.department}">HR</td>
          <td th:text="${e.salary}">50000</td>
          <td>
            <a class="btn btn-sm btn-warning" th:href="@{/employees/${e.id}/edit}">Edit</a>
            <form th:action="@{/employees/${e.id}/delete}" method="post" class="d-inline">
              <button class="btn btn-sm btn-danger" onclick="return
confirm('Delete?')">Delete</button>
            </form>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</body>
```

```

</html>
templates/employee-form.html (Add/Edit Form)
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title th:text="${title}">Employee Form</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
</head>
<body class="p-4">
  <div class="container">
    <h2 th:text="${title}">Add Employee</h2>

    <form th:action="@{/employees}" th:object="${employee}" method="post" class="mt-3">
      <input type="hidden" th:field="*{id}">

      <div class="mb-3">
        <label class="form-label">Name</label>
        <input class="form-control" th:field="*{name}">
        <div class="text-danger" th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></div>
      </div>

      <div class="mb-3">
        <label class="form-label">Email</label>
        <input class="form-control" th:field="*{email}">
        <div class="text-danger" th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></div>
      </div>

      <div class="mb-3">
        <label class="form-label">Department</label>
        <input class="form-control" th:field="*{department}">
        <div class="text-danger" th:if="${#fields.hasErrors('department')}"
th:errors="*{department}"></div>
      </div>

      <div class="mb-3">
        <label class="form-label">Salary</label>
        <input class="form-control" th:field="*{salary}" type="number" step="0.01">
        <div class="text-danger" th:if="${#fields.hasErrors('salary')}"
th:errors="*{salary}"></div>
      </div>

      <button class="btn btn-success" type="submit">Save</button>
      <a class="btn btn-secondary" th:href="@{/employees}">Cancel</a>
    </form>
  </div>

```

```
</body>  
</html>
```

7) Main App – EmployeeCrudApplication.java

```
package com.example.employeecrud;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class EmployeeCrudApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(EmployeeCrudApplication.class, args);  
    }  
}
```

How to Run

1. Start **MySQL** and update username/password in `application.properties`.
2. In project folder: `mvn spring-boot:run` (or run from IDE).
3. Open `http://localhost:8080/employees`.
 - + **Add Employee** → Create
 - **Edit** button → Update
 - **Delete** button → Delete
 - List page → Read all

Output:

Spring Boot MVC CRUD

Employee Records

[+ Add Employee](#)

ID	Name	Email	Department	Salary	Actions
1	John Doe	john@example.c	IT	50000	Edit Delete
2	Jane Smith	jane@example.c	HR	60000	Edit Delete
3	Bob Johnson	bob@example.c	Marketing	45000	Edit Delete
4	Mark Lee	mark@example.c	Sales	55000	Edit Delete

Add Employee

Name

Email

Department

Salary

[Create](#)

RESULT:

The Spring Boot Employee Management CRUD application with MySQL integration allows users to:

1. **Create** – Add new employee records with details like name, email, and department.
2. **Read** – View a list of employees stored in the database.
3. **Update** – Edit existing employee information.
4. **Delete** – Remove employee records from the database.

The result is a fully functional **web-based Employee Management System** where employee data is persistently stored in **MySQL** and managed via **Spring Boot MVC + JPA/Hibernate**.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3

Ex. No. : 10

Create a web application that integrates MongoDB for real-time data visualization using Spring Boot.

Aim:

To develop a Spring Boot web application that integrates **MongoDB** and provides **real-time data visualization** in the browser using **SSE** and **Chart.js**.

Procedure:

Step 1: Create a Spring Boot project with dependencies: *Spring WebFlux, Spring Data MongoDB Reactive*.

Step 2: Configure MongoDB connection in `application.properties`.

Step 3: Define a domain model (e.g., `SensorReading`: id, timestamp, value).

Step 4: Create a **Reactive Mongo Repository** for CRUD.

Step 5: Implement a **Change Stream** listener and expose it as **SSE** (`text/event-stream`).

Step 6: Build a web page (`index.html`) that uses **Chart.js** and **EventSource** to stream points into a live chart.

Step 7: (Optional) Add a data ingestor endpoint or a scheduled inserter to simulate new readings.

Step 8: Run the app, open the page, and watch the chart update in real time.

Program:

1) pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>realtime-mongo-visualization</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.2</version>
  </parent>

  <dependencies>
    <!-- Reactive Web + Mongo -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
    </dependency>
  </dependencies>
</project>
```

```

<!-- Optional: validation -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Lombok (optional) -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

2) src/main/resources/application.properties

spring.application.name=realtime-mongo-visualization

MongoDB (use your Atlas URI or local replica set)

spring.data.mongodb.uri=mongodb://localhost:27017/realtime?replicaSet=rs0

server.port=8080

spring.webflux.static-path-pattern=/**

3) Domain model – SensorReading.java

package com.example.realtime.model;

import org.springframework.data.annotation.Id;

import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;

```
@Document("sensor_readings")
public class SensorReading {
    @Id
    private String id;
    private Instant timestamp;
    private double value;

    public SensorReading() {}
    public SensorReading(Instant timestamp, double value) {
        this.timestamp = timestamp;
        this.value = value;
    }
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public Instant getTimestamp() { return timestamp; }
    public void setTimestamp(Instant timestamp) { this.timestamp = timestamp; }
    public double getValue() { return value; }
    public void setValue(double value) { this.value = value; }
}
```

4) Repository – SensorReadingRepository.java

```
package com.example.realtime.repo;

import com.example.realtime.model.SensorReading;
import org.springframework.data.mongodb.repository.ReactiveMongoRepository;

public interface SensorReadingRepository extends ReactiveMongoRepository<SensorReading,
String> { }
```

5) SSE Controller with Change Streams – StreamController.java

```
package com.example.realtime.web;

import com.example.realtime.model.SensorReading;
import com.mongodb.client.model.changestream.ChangeStreamDocument;
import org.bson.Document;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.mongodb.core.ChangeStreamEvent;
import org.springframework.data.mongodb.core.ReactiveMongoTemplate;
import org.springframework.data.mongodb.core.messaging.ChangeStreamRequest;
import org.springframework.data.mongodb.core.messaging.Message;
import org.springframework.data.mongodb.core.messaging.ReactiveMongoMessageListenerContainer;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
```

```
import java.time.Instant;
import java.util.Map;

@Controller
public class StreamController {

    private final ReactiveMongoTemplate template;
    private final ReactiveMongoMessageListenerContainer container;

    @Autowired
    public StreamController(ReactiveMongoTemplate template) {
        this.template = template;
        this.container = new ReactiveMongoMessageListenerContainer(template);
    }

    // SSE stream of new SensorReading changes
    @GetMapping(path = "/stream/readings", produces =
    MediaType.TEXT_EVENT_STREAM_VALUE)
    @ResponseBody
    public Flux<Map<String, Object>> streamReadings() {
        ChangeStreamRequest<ChangeStreamDocument<Document>> request =
            ChangeStreamRequest.builder()
                .collection("sensor_readings")
                .filter(Criteria.where("operationType").in("insert", "update", "replace"))
                .build();

        Flux<Message<ChangeStreamDocument<Document>, ChangeStreamEvent<Document>>>
        flux =
            container.register(request, Document.class);

        return flux
            .map(msg -> msg.getBody().getFullDocument())
            .filter(doc -> doc != null)
            .map(doc -> Map.of(
                "timestamp", doc.get("timestamp"),
                "value", doc.get("value")
            ));
    }

    // Quick insert API to simulate data (POST /api/readings?value=42.5)
    @PostMapping("/api/readings")
    @ResponseBody
    public Map<String, Object> addReading(@RequestParam double value) {
        SensorReading r = new SensorReading(Instant.now(), value);
        return template.save(r).map(saved -> Map.of(
            "id", saved.getId(),
            "timestamp", saved.getTimestamp(),
```

```

    "value", saved.getValue()
  })).block(); // for demo simplicity
}

```

```

// Serve the index page
@GetMapping("/")
public String index() { return "forward:/index.html"; }
}

```

If you prefer a simpler approach without the message container, you can also use `template.changeStream(SensorReading.class)` to get a `Flux<ChangeStreamEvent<SensorReading>>` and map it to SSE. The above uses the **message listener container** for clarity.

6) Frontend – `src/main/resources/static/index.html`

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Real-time MongoDB Chart</title>
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
  <style>
    body { font-family: system-ui, Arial, sans-serif; margin: 20px; }
    .wrap { max-width: 900px; margin: auto; }
    canvas { width: 100%; height: 420px; }
    .toolbar { display: flex; gap: 8px; margin-bottom: 12px; }
    input, button { padding: 8px 10px; }
  </style>
</head>
<body>
  <div class="wrap">
    <h2>Real-time Sensor Readings (MongoDB → SSE → Chart.js)</h2>
    <div class="toolbar">
      <input id="val" type="number" step="0.1" placeholder="Enter value e.g. 25.6" />
      <button id="send">Insert Reading</button>
      <span id="status">Connecting...</span>
    </div>
    <canvas id="chart"></canvas>
  </div>

  <script>
    const ctx = document.getElementById('chart');
    const labels = [];
    const data = { labels, datasets: [{ label: 'Value', data: [] }] };
    const chart = new Chart(ctx, {
      type: 'line',
      data,

```

```

    options: {
      animation: false,
      scales: { x: { ticks: { maxRotation: 0, autoSkip: true } } }
    }
  });

  // SSE connection
  const es = new EventSource('/stream/readings');
  const status = document.getElementById('status');
  es.onopen = () => status.textContent = 'Live connected';
  es.onerror = () => status.textContent = 'Disconnected (retrying...)';
  es.onmessage = (e) => {
    try {
      const { timestamp, value } = JSON.parse(e.data);
      // label: timestamp string, data point: value
      labels.push(new Date(timestamp).toLocaleTimeString());
      data.datasets[0].data.push(value);
      if (labels.length > 50) { labels.shift(); data.datasets[0].data.shift(); }
      chart.update();
    } catch (err) {
      // For container mapping we may receive raw maps; handle both.
      const obj = typeof e.data === 'string' ? e.data : "";
    }
  };

  // Quick insert for demo
  document.getElementById('send').onclick = async () => {
    const v = parseFloat(document.getElementById('val').value);
    if (isNaN(v)) return alert('Enter a numeric value');
    await fetch('/api/readings?value=' + encodeURIComponent(v), { method: 'POST' });
    document.getElementById('val').value = "";
  };
</script>
</body>
</html>

```

7) (Optional) Data Seeder – DemoDataRunner.java

```

package com.example.realtime.config;

import com.example.realtime.model.SensorReading;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.mongodb.core.ReactiveMongoTemplate;
import reactor.core.publisher.Flux;

import java.time.Duration;

```

```
import java.time.Instant;
import java.util.Random;

@Configuration
public class DemoDataRunner {
    @Bean
    CommandLineRunner seed(ReactiveMongoTemplate template) {
        return args -> {
            Random rnd = new Random();
            Flux.interval(Duration.ofSeconds(2))
                .flatMap(t -> template.save(new SensorReading(Instant.now(), 20 + rnd.nextDouble() *
10)))
                .subscribe();
        };
    }
}
```

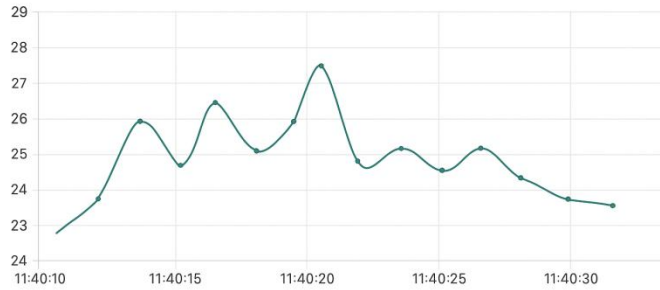
(This continuously inserts random readings every 2 seconds so you can see the chart move.)

How to Run

1. **Start MongoDB as a replica set** (or use MongoDB Atlas):
 - Local: `mongod --replSet rs0 --dbpath <path> → mongosh → rs.initiate()`.
2. Update `spring.data.mongodb.uri` if needed.
3. Run the app: `mvn spring-boot:run`.
4. Open **`http://localhost:8080/`** in your browser.
5. Watch the chart update automatically; optionally type a value and click **Insert Reading**.

Output:**Real-time Sensor Readings (MongoDB → SSE → Chart.js)**

Live connected



Result:

A Spring Boot application successfully integrates with **MongoDB Change Streams** to provide **real-time visualization** of data in the browser using **SSE** and **Chart.js**. CRUD operations (inserts/updates) in MongoDB are instantly reflected on the UI, demonstrating end-to-end streaming from database to client.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3

Content Beyond the Syllabus

Ex.No.: 11 Angular Js Program For Payroll Management System

AIM:

Write an angular JS program for payroll management system which includes the modules of admin can manage the salary, admin can edit/delete the salary, admin can see the list of all salary, employee can see his salary.

ALGORITHM

1. Design the JS Program Payroll Management system with the following
2. Create a Module of admin page
3. Create Payroll management Employee details.

PROGRAM

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
$scope.filteredDept = [];
$scope.employees = [
{'id':1, 'dept': 'IT', 'sal': 5000, 'name': 'ramesh'},
{'id':2, 'dept': 'IT', 'sal': 4000, 'name': 'Girish'},
{'id':3, 'dept': 'Management', 'sal': 10000, 'name': 'Kartik'},
{'id':4, 'dept': 'Marketing', 'sal': 8000, 'name': 'Shenoy'},
{'id':5, 'dept': 'Marketing', 'sal': 7000, 'name': 'Darvik'},
{'id':6, 'dept': 'Management', 'sal': 17000, 'name': 'Ravi Kumar'}
];
$scope.empFiltered = $scope.employees;
$scope.departments = [
{'id':1, 'name':'IT','selected':false},
{'id':2, 'name':'Management','selected':false},
{'id':3, 'name':'Marketing','selected':false}
];
$scope.sortEmp = function(dept_id){
$scope.bool = $scope.departments.filter(function(e){
if(e.id === dept_id){
var idx = $scope.filteredDept.indexOf(e.name,0);
if(idx === -1){
$scope.filteredDept.push(e.name);
$scope.filterEmpList();
return 0;
}
else {
$scope.filteredDept.splice(idx,1);
$scope.filterEmpList();
return 0;
}
}
```

IFETCE R-2019 ACADEMIC YEAR: 2022-2023

IFET COLLEGE OF ENGINEERING| Department of Computer Science & Engineering 14

```

    }
  });
};
$scope.filterEmpList = function(){
$scope.filteredData = [];
if($scope.filteredDept.length !==0){
$scope.empFiltered = [];
angular.forEach($scope.filteredDept, function(val,key){
$scope.filteredData = $scope.employees.filter(function(e){
  if(e.dept === val){
    return e;
  }
});
angular.forEach($scope.filteredData, function(v,k){
$scope.empFiltered.push(v);
});
});
}
else{
$scope.empFiltered = $scope.employees;
}
};
$scope.hikeSal = function(){
var arr = [];
arr = $scope.empFiltered.map(function(e){
if(e.hike === undefined){
e.hike = 0;
}
e.hike = parseInt(e.hike) + $scope.hike;
e.hikeAmt = e.sal * ($scope.hike * 0.01);
e.sal += e.sal * ($scope.hike * 0.01);
//return e;
});
});
});
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.4/angular.min.js"></script>
<body>
<div ng-app="myApp" ng-controller="myCtrl">
percentage : <input type="number" ng-model="hike"/>
<button ng-click="hikeSal()">Hike the salary</button>
<br>
Hike by dept : <br>
<div class="checkbox" ng-repeat="dept in departments track by $index">

```

```

<label>
IFETCE R-2019 ACADEMIC YEAR: 2022-2023
IFET COLLEGE OF ENGINEERING| Department of Computer Science & Engineering 15
<input type="checkbox" ng-model="dept.selected" ng-click="sortEmp(dept.id)"/>
{{dept.name}}
</label>
</div>
<table border='1'>
<thead>
<tr>
<th>Name</th>
<th>Dept</th>
<th>Hike Percentage</th>
<th>Hike Amount</th>
<th>Salary</th>
</tr>
<tr ng-repeat="emp in empFiltered track by $index">
<td>{{emp.name}}</td>
<td>{{emp.dept}}</td>
<td>{{emp.hike}}</td>
<td>{{emp.hikeAmt}}</td>
<td>{{emp.sal | currency: '&#8360;'}}</td>
</tr>
</thead>
</table>
</div>
</body>

```

OUTPUT

Percentage : Hike the salary

Hike by dept :

☐ IT☒ Management☐ Marketing

Name	Dept	Hike Percentage	Hike Amount	Salary
ramesh	IT	5	250	Rs 5,250.00
Girish	IT	5	200	Rs 4,200.00
Kartik	Management	5	500	Rs 10,500.00
Shenoy	Marketing	5	400	Rs 8,400.00
Darvik	Marketing	5	350	Rs 7,350.00
Ravi Kumar	Management	5	850	Rs 17,850.00

RESULT

Thus, we have create angular JS program for payroll management system
On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3

Ex No: 12 IMPLEMENT A WEB APPLICATION USING STRUTS FRAMEWORK WHICH CONTAINS LOGIN INFORMATION**AIM**

To implement a web application using Struts framework with login information.

PROCEDURE:

- Create login.jsp which will contain the form to be displayed to the user.
- Create a Form Bean Class (LoginForm.java) that will hold the form values provided by the user.
- Create a Controller class (LoginAction.java) that will check for the user input and decide which view to be redirected to the users either success.jsp or failure.jsp
- Create a success page (Success.jsp) and failure page (Failure.jsp)
- And finally will configure our form bean and action classes in struts-config.xml.

PROGRAM**Files Required**

Following files are required for this application.

- login.jsp
- success.jsp
- LoginForm.java
- LoginAction.java
- struts-config.xml
- web.xml

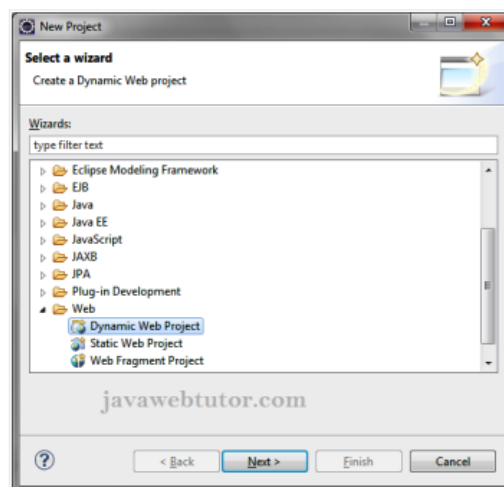
Tools used:

In order to create an application we are going to use the following tools.

- JDK 1.5 or above
- Eclipse Indigo or above
- Tomcat 6.x above or any other server which supports java like jboss, weblogic, glassfish
- Struts 1.2 JAR files

Step 1: Create Dynamic Web Project

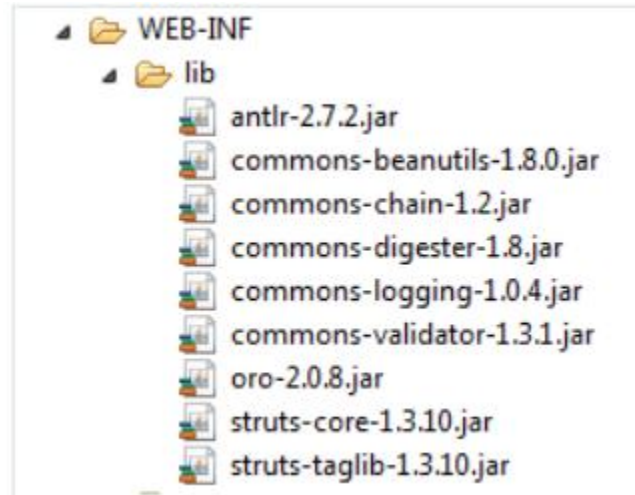
Open Eclipse and goto File -> New -> Project and select Dynamic Web Project in the New Project wizard screen.



After selecting Dynamic Web Project, press Next Then Eclipse will ask for name of the project. Enter the name of the project as LoginExampleStruts1 and click on Finish.

Step 2: Add Jar files to the project

Now copy all the required JAR files in WebContent -> WEB-INF -> lib folder. The following jar files should be added to the project for successful deployment of struts project:



After that, have to configure ActionServlet of struts with web.xml. The first page that will be called in the login application is the login.jsp page. This configuration should be done in web.xml as <welcome-file-list>. The following xml shows how to configure struts in web.xml.

Step 3: Configure web.xml

Now have to configure ActionServlet of struts with web.xml. The following xml shows how to configure struts in web.xml.

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaeehttp://java.sun.com
/xml/ns/javaee/web-app_2_5.xsd"
id="WebApp_ID" version="2.5">
<display-name>LoginFormStruts1</display-name>
<servlet>
<servlet-name>action</servlet-name>
<servlet-class>org.apache.struts.action.ActionServlet</servlet-class>
<init-param>
<param-name>config</param-name>
<param-value>
/WEB-INF/struts-config.xml
</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
```

```
<servlet-name>action</servlet-name>
<url-pattern>*.do</url-pattern>
</servlet-mapping>
<welcome-file-list>
<welcome-file>index.jsp</welcome-file>
</welcome-file-list></web-app>
```

Step 4: Create FormBean Class:

LoginForm.java

```
package com.example.javawebtutor.form;
import javax.servlet.http.HttpServletRequest;
import org.apache.struts.action.ActionForm;
import org.apache.struts.action.ActionMapping;
public class LoginForm extends ActionForm {
    private String userName = null;
    private String password = null;
    public String getUserName() {
        return userName;
    }
    public void setUserName(String userName) {
        this.userName = userName;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    @Override
    public void reset(ActionMapping mapping, HttpServletRequest request) {
        this.password = null;
    }
}
```

There are two fields in this form bean "userName" and "password", that will hold the value of two fields userName and password entered by user. To override the method "reset" in form bean. "reset" method is called at the end of the every request processed by the struts. In reset method set the value of password as null which means every time the user will open the login.jsp in browser it will show the last entered username and reset the password field.

Step 5: Create Action Class:

Following is our controller helper class named as "LoginAction":

LoginAction.java

```
package com.example.javawebtutor.action;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.apache.struts.action.Action;
import org.apache.struts.action.ActionForm;
import org.apache.struts.action.ActionForward;
import org.apache.struts.action.ActionMapping;
import com.example.javawebtutor.form.LoginForm;
```

```

public class LoginAction extends Action {
    @Override
    public ActionForward execute(ActionMapping mapping, ActionForm form,
        HttpServletRequest request, HttpServletResponse response) throws
        Exception {
        LoginForm loginForm = (LoginForm) form;
        if (loginForm.getUserName() == null || loginForm.getPassword() == null
            || !loginForm.getUserName().equalsIgnoreCase("Mukesh")
            || !loginForm.getPassword().equals("kumar")) {
            return mapping.findForward("success");
        } else
            return mapping.findForward("failure"); } }

```

In this class ActionForm instance is typecast to LoginForm in the execute method of LoginAction and then logic to verify the username and password will decide which view to be send back to the user. In this case, userName must be "Mukesh" and password must be "kumar" to go to the view associated with the "success", otherwise view associated with "failure" will be returned.

These configurations have been done in struts-config.xml file. The entire configuration regarding the action will be made as in the file struts-config.xml mentioned below.

Step 6: Create struts-config.xml file

Create struts-config.xml inside WEB-INF directory of project and add below line of code in this file.

```

struts-config.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE struts-config PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 1.3//EN"
"http://struts.apache.org/dtds/struts-config_1_3.dtd">
<struts-config>
<form-beans>
<form-bean name="loginForm" type="com.example.javawebtutor.form.LoginForm"/>
</form-beans>
<action-mappings>
<action name="loginForm" path="/login"
type="com.example.javawebtutor.action.LoginAction" scope="request" input="/login.jsp">
<forward name="failure" path="/login.jsp" redirect="true"/>
<forward name="success" path="/success.jsp" redirect="true"/>
</action>
</action-mappings>
</struts-config>

```

Step 7: Create jsp files

Create jsp file login.jsp inside WebContent directory of your project with following contents.

```

login.jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://struts.apache.org/tags-html" prefix="html"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

```

```

<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Login Example</title>
</head>
<body>
<html:form action="/login" focus="userName">
Username : <html:text property="userName" />
<br>
Password : <html:password property="password" />
<br>
<html:submit value="login" />
</html:form>
</body></html>

```

In login.jsp, Struts HTML Tags are used to create login page. The form has two text fields to get the user name and password. The form also has one submit button, which when clicked calls the login action. <html:errors/> tag is used to display the error messages to the user. Now create jsp file success.jsp inside WebContent directory of project with following contents.

```

success.jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Successful Login Page</title>
</head>
<body>
<h2>Welcome Boss</h2>
</body>
</html>

```

User will navigate to success.jsp page after successful login. Now create jsp file failure.jsp inside WebContent directory of project with following contents.

```

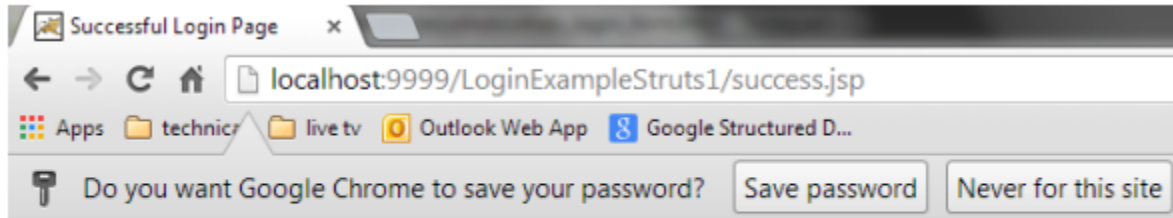
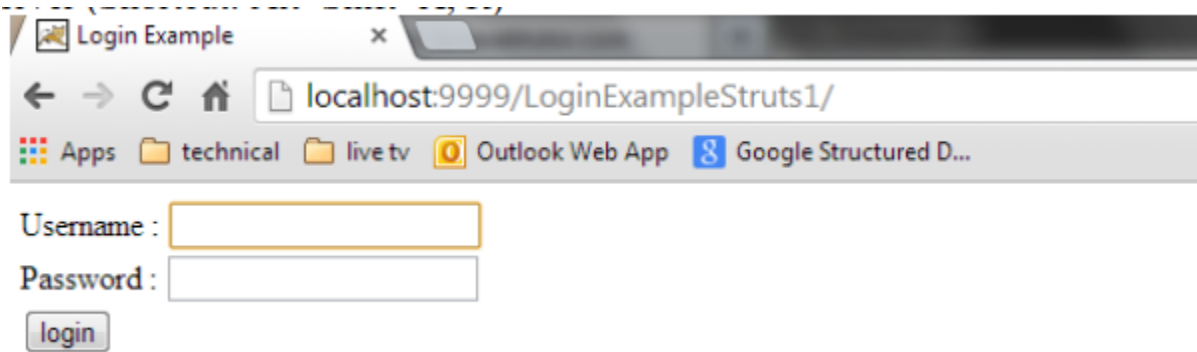
index.jsp
<jsp:forward page="/login.jsp"></jsp:forward>

```

Step 10: Run the application:

To run the project, right click on Project name from Project Explorer and select Run as -> Run on Server (Shortcut: Alt+Shift+X, R)

Now, Enter "Mukesh" as username and "Kumar" as password you will be carried out to the success.jsp page



Welcome Boss

RESULT:

Thus, the development of web application using struts framework for login page was created successfully

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3

EX.13 STORES A FORM DATA INTO DATABASE USING PHP**AIM**

To store a form data into database using php.

PROCEDURE:

1. Create a form which needs an user information
2. Get the form information from index page into insert page
3. Insert the data into the database
4. If the data was successfully inserted, provide the information as new record created successfully.
5. Otherwise the form will throw appropriate error.

SOURCE CODE**index.php:**

```
<html>
<body>
<h1>A example page to insert some data in to the MySQL database using PHP</h1>
<form action="insert.php" method="post">
Firstname: <input type="text" name="fname" /><br><br>
Lastname: <input type="text" name="lname" /><br><br>
Email ID: <input type="text" name="email" /><br><br>
<input type="submit" />
</form>
</body>
</html>
```

insert.php

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('$_POST[fname]', '$_POST[lname]', '$_POST[email]')";
if ($conn->query($sql) === TRUE) {
```

```

    echo "New record created successfully";
}
else
{
    echo "Error: " . $sql . "<br>" . $conn->error;
}
$conn->close();
?>

```

OUTPUT:

A example page to insert some data in to the MySQL database using PHP

Firstname:

Lastname:

Email ID:

New record created successfully

RESULT:

Thus the data from the form has been saved into the database using php was implemented successfully.

On completion of the above program the following PO's PSO's are attained.

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO5	3	3	3	3	3		1	2	1	2	1	2	3	3	3